

DOORS

End-to-End Encryption and Decryption Process

Machine/API-client cryptographic exchange, authorization gates and protected response handling

Scope.

Defines the supported encrypted request and response contracts between a registered API Client and the Master API. Browser-session response protection is intentionally documented separately. HTTPS/TLS is mandatory for every hop.

1. Architecture and trust boundaries

API Client -> TLS -> API plane -> Master API -> authorized agent/data source -> Master API -> TLS -> API Client

- The API Client owns its private signing/decryption key; DOORS stores only the registered Client public key.
- The Master API owns its active private unwrap/signing keys and publishes the corresponding trusted public keys.
- Authorization, Client-to-query mapping, agent scope, rate limits and audit checks occur independently of encryption.
- Cryptographic failure is terminal: no plaintext fallback and no partial business response.

2. Inbound encrypted request contract

<pre>{ "encryptedKey": "<Base64 RSA-wrapped AES key>", "iv": "<Base64 IV>", "secureData": "<Base64 ciphertext>", "signature": "<Base64 Client signature>" }</pre>		
Field	Protection	Purpose
encryptedKey	RSA/ECB/PKCS1Padding	Fresh AES session key wrapped with the active DOORS public key
iv	Base64; required for CBC	Initialization vector associated with this request
secureData	AES encrypted UTF-8 JSON	Request body containing agentIds and params
signature	SHA512withRSA; configured SHA256 fallback	Authenticates the exact raw plaintext JSON before parsing

3. Client request-protection sequence

1. **Prepare.** Construct the minimum authorized JSON request and serialize it once to exact UTF-8 bytes.
2. **Sign.** Sign the exact raw JSON bytes with the Client private key.
3. **Generate.** Create a fresh AES key and IV for this request.
4. **Encrypt.** Encrypt the exact raw JSON using the configured AES mode; current gateway support accepts CBC with IV and the documented compatibility mode.
5. **Wrap.** Encrypt the AES key with the current DOORS RSA public key using RSA/ECB/PKCS1Padding.
6. **Send.** Base64-encode binary fields and transmit the wrapper over validated TLS with the registered API credential.

4. Master API inbound processing gate

1. **Authenticate.** Resolve the API Client, validate credential state, IP constraints, rate limits and registered certificate/key profile.
2. **Unwrap.** Decrypt encryptedKey with the active Master API private key.
3. **Decrypt.** Validate the IV/mode and decrypt secureData to the exact raw UTF-8 JSON.
4. **Verify.** Verify signature using the registered Client public key before parsing or execution.

5. **Authorize.** Validate query/client mappings, requested agentIds, parameters and data-access policy.
6. **Execute.** Only after all gates succeed, orchestrate the approved operation and record the trace/audit outcome.

5. Outbound encrypted response contract

For an encryption-enabled API Client, the Master API projects only approved fields, serializes the exact response JSON, signs it, encrypts it with a fresh AES-256 key and IV, then wraps that key with the Client public RSA key.

<pre>{ "encryptedKey": "<Base64 Client-wrapped AES-256 key>", "iv": "<Base64 16-byte IV>", "secureData": "<Base64 AES-CBC ciphertext>", "signature": "<Base64 SHA256withRSA signature>" }</pre>		
Item	Current contract	Verification
Payload	AES-256/CBC/PKCS5Padding	Decrypt with unwrapped AES key and exactly 16-byte IV
Key wrap	RSA/ECB/PKCS1Padding	Unwrap only with the Client private key
Signature	SHA256withRSA over exact raw response JSON	Verify using trusted DOORS public key selected by X-DOORS-KID
Headers	X-Content-Secure, X-DOORS-KID, X-DOORS-TRACE	Require secure marker and retain trace ID for diagnostics

6. API Client response-processing sequence

1. **Validate envelope.** Require encryptedKey, iv and secureData; validate Base64 and the secure response marker.
2. **Unwrap key.** Use the Client private key to decrypt encryptedKey; require a valid AES-256 key.
3. **Decrypt payload.** Decode the 16-byte IV and decrypt secureData with AES/CBC/PKCS5Padding (PKCS7 is the equivalent library label).
4. **Verify signature.** Verify the exact decrypted UTF-8 JSON before parsing or normalization.
5. **Parse and use.** Validate status/message/payload shape, then expose only the authorized result to the application.

6. **Fail closed.** On any key, padding, signature, UTF-8 or JSON error, discard the result and report only trace-safe diagnostics.

7. Operational controls and acceptance checks

- Fresh AES key and IV are generated for every encrypted request/response; neither is reused.
- Private keys never cross their owning trust boundary and are never written to logs.
- The signature input is the exact raw JSON byte sequence; reserialization before verification is prohibited.
- TLS certificate chain and hostname validation remain mandatory; payload encryption does not replace TLS.
- Negative tests cover altered ciphertext, signature mismatch, invalid IV, stale key ID, unauthorized agent/query and revoked Client.
- Audit records include Client identity, operation, agent scope, outcome, correlation/trace ID and timestamp - never keys or plaintext.

8. Implementation references

Responsibility	Current implementation
Gateway decrypt/verify	ExternalGatewayController encrypted-wrapper processing
Response protection	ApiClientResponseEncryptionService
Published contract	DynamicOrchestrationOpenApiCustomizer and TemplateContractService
Client examples	Generated Java SDK/curl guidance from the API Client registry

End of controlled document.